Tecnologia Java: Introdução.

Assuntos abordados

- Conceitos
  - Tecnologia Java
  - Linguagem e API Java
  - Máquina Virtual
  - Ambiente de execução (JRE) e desenvolvimento (SDK)
  - Carregador de classes (Classloader) e CLASSPATH
  - Verificador de bytecodes
  - Coletor de lixo (garbage collector)

Introdução Prática: como escrever uma aplicação, compilar, executar e depurar erros de compilação e execução.

Tecnologia Java

- *O nome "Java" é usado para referir-se a*
  - *Uma **linguagem de programação orientada a objetos***
  - *Uma coleção de **APIs (classes, componentes, frameworks) para o** desenvolvimento de aplicações multiplataforma*
  - *Um **ambiente de execução presente em browsers, mainframes,** SOs, celulares, palmtops, cartões inteligentes, eletrodomésticos*
- *Java foi lançada pela Sun em 1995. Três grandes revisões*
  - *Java Development Kit (JDK) 1.0/1.0.2*
  - *Java Development Kit (JDK) 1.1/1.1.8*
  - *Java 2 Platform (Java 2 SDK e JRE 1.2, 1.3, 1.4)*
  - *JDK 5.0, 6.0, 7.0 e 8.0.*
- *A evolução da linguagem é controlada pelo **Java Community Process (www.jcp.org)***
- *Ambientes de execução e desenvolvimento são fornecidos por fabricantes de hardware e software (MacOS, Linux, etc.)*

Linguagem Java

- *Linguagem de programação orientada a objetos*
  - *Familiar (sintaxe parecida com C)*
  - *Simples e robusta (minimiza bugs, aumenta produtividade)*
  - *Suporte nativo a threads (+ simples, maior portabilidade)*
  - *Com coleta de lixo (menos bugs, mais produtividade)*
  - *Independente de plataforma*
  - *Segura (vários mecanismos para controlar segurança)*
  - *Código intermediário de máquina virtual interpretado*
  - *Sintaxe uniforme, rigorosa quanto a tipos (código mais simples, menos diferenças em funcionalidades iguais)*

Produtos e APIs

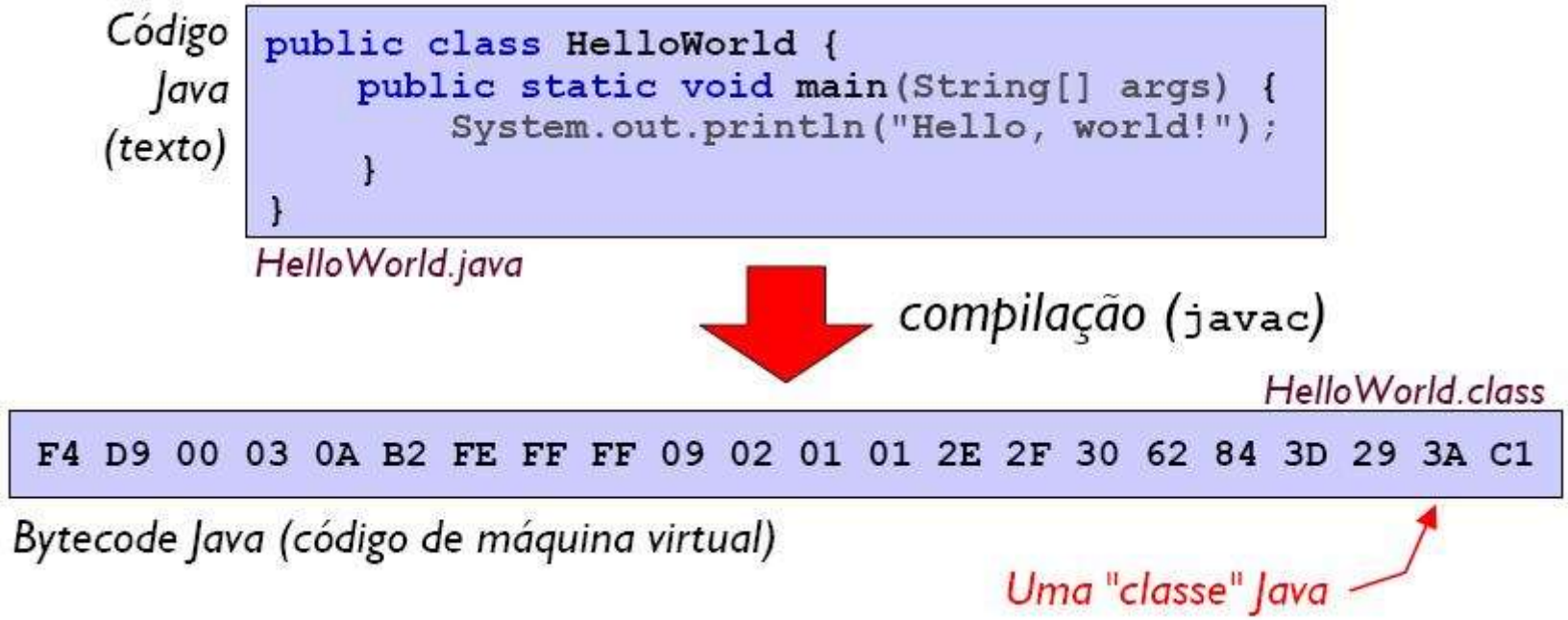
- *Java possui uma coleção de APIs (bibliotecas) padrão que podem ser usadas para construir aplicações*
  - *Organizadas em* ***pacotes (java.*, javax.* e extensões)***
  - *Usadas pelos ambientes de execução (**JRE) e de** desenvolvimento (**SDK)***
- *As principais APIs são distribuídas juntamente com os produtos para desenvolvimento de aplicações*
  - ***Java Standard Edition (JSE): ferramentas e APIs** essenciais para qualquer aplicação Java (inclusive GUI)*
  - ***Java Enterprise Edition (JEE): ferramentas e APIs** para o desenvolvimento de aplicações distribuídas*
  - ***Java Micro Edition (JME): ferramentas e APIs para o** desenvolvimento de aplicações para aparelhos portáteis*

Ambiente de execução e desenvolvimento

- ***Java Development Kit (JDK)***
  - *Coleção de ferramentas de linha de comando para, entre outras tarefas, compilar, executar e depurar aplicações Java*
  - *Para habilitar o ambiente via linha de comando é preciso colocar o caminho* ***`$JAVA_HOME/bin`*** no PATH do sistema
- ***Java Runtime Environment (JRE)***
  - *Tudo o que é necessário para executar aplicações Java*
  - *Parte do JDK e das principais distribuições Linux, MacOS X, AIX, Solaris, Windows 98/ME/2000 (exceto XP)*
- *Variável* **JAVA_HOME** *(opcional: usada por vários frameworks)*
- *Defina com o local de instalação do Java no seu sistema.*
  - *Exemplos:*
    - *Windows:* ***set JAVA_HOME=c:\jdk1.5.0***
    - *Linux:* ***JAVA_HOME=/usr/java/jdk1.5.0***
      - **export JAVA_HOME**

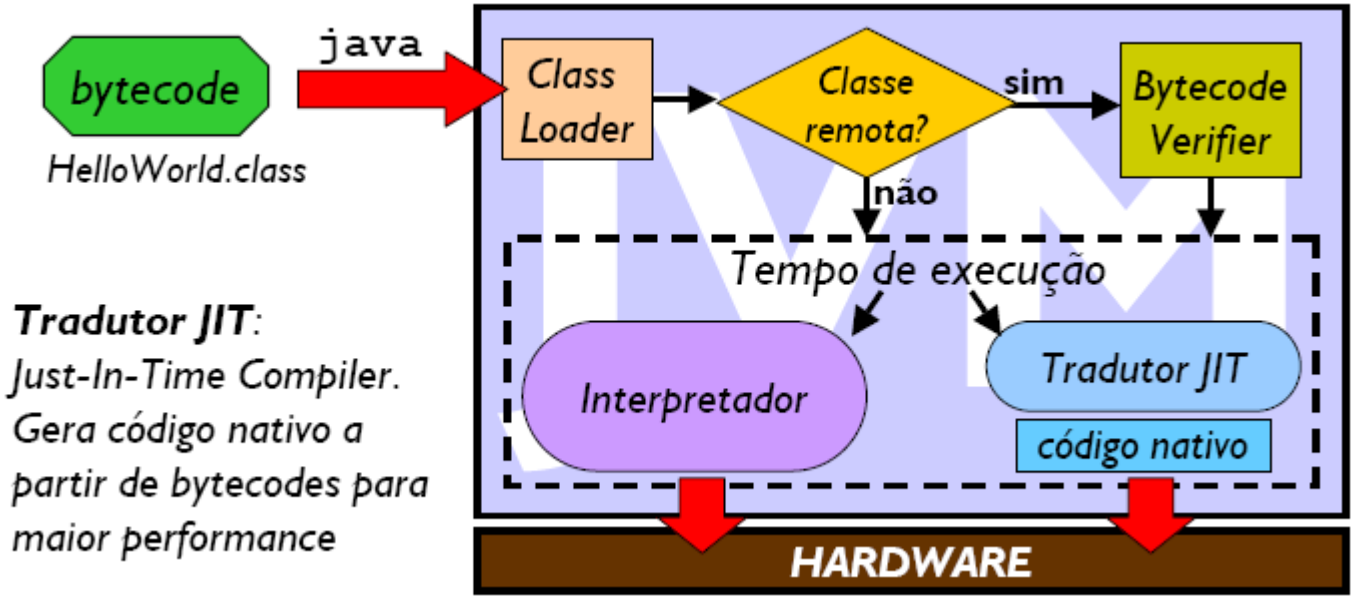
Compilação para bytecode

- **Bytecode é o código de máquina que roda em qualquer** máquina através da **Máquina Virtual Java (JVM)**
- Texto contendo código escrito em linguagem Java é traduzido em bytecode através do processo de **compilação e** armazenado em um arquivo ***.class chamado de Classe Java**



Máquina Virtual Java (JVM)

- *"Máquina imaginária implementada como uma aplicação de software em uma máquina real"*
- *A forma de execução de uma aplicação depende ...*
  - *... da origem do código a ser executado (remoto ou local)*
  - *... da forma como foi implementada a JVM pelo fabricante (usando tecnologia JIT, HotSpot, etc.)*



Tradutor JIT:
Just-In-Time Compiler. Gera código nativo a partir de bytecodes para maior performance

Class Loader e CLASSPATH

- *Primeira tarefa executada pela JVM: carregamento das classes necessárias para rodar a aplicação. O **Class Loader***
  - *1. Carrega primeiro as classes nativas do **JRE (APIs)***
  - *2. Depois carrega extensões do **JRE:** JARs em* `$JAVA_HOME/jre/lib/ext` *e classes em* `$JAVA_HOME/jre/lib/classes`
  - *3. Carrega classes do sistema local (a ordem dos caminhos no **CLASSPATH** define a precedência)*
  - *4. Por último, carrega possíveis classes remotas*
- **CLASSPATH:** *variável de **ambiente local** que contém todos os caminhos locais onde o Class Loader pode localizar classes*
  - *A CLASSPATH é lida depois, logo, suas classes nunca substituem as classes do JRE (não é possível tirar classes JRE do CLASSPATH)*
  - *Classes remotas são mantidas em área sujeita à verificação*
  - *CLASSPATH pode ser redefinida através de parâmetros durante a execução do comando* **java**

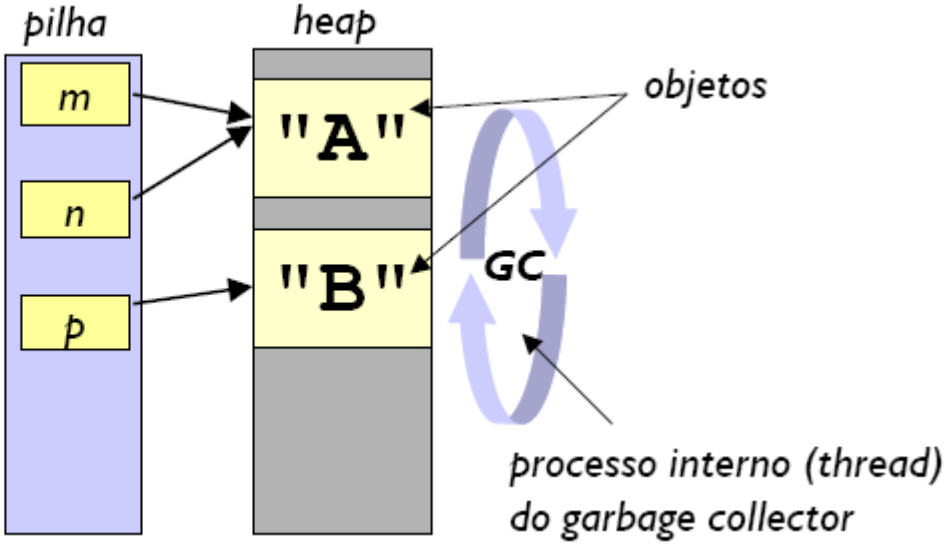
Bytecode Verifier

- *Etapa que antecede a execução do código em classes carregadas através da rede*
  - *Class Loader distingue classes locais (seguras) de classes remotas (potencialmente inseguras)*
- *Verificação garante*
  - *Aderência ao formato de arquivo especificado*
  - *Não-violação de políticas de acesso estabelecidas pela aplicação*
  - *Não-violação da integridade do sistema*
  - *Ausência de estouros de pilha*
  - *Tipos de parâmetros corretamente especificados e ausência de conversões ilegais de tipos*

Coleta de lixo

- *Memória alocada em Java não é liberada pelo programador*
  - *Ou seja, objetos criados não são destruídos pelo programador*
- *A criação de objetos em Java consiste de*
  - *1. Alocar memória no heap para armazenar os dados do objeto*
  - *2. Inicializar o objeto (via construtor)*
  - *3. Atribuir endereço de memória a uma variável (referência)*
- *Mais de uma referência pode apontar para o mesmo objeto*

```
Mensagem m, n, p;
m = new Mensagem("A");
n = m;
p = new Mensagem("B");
```

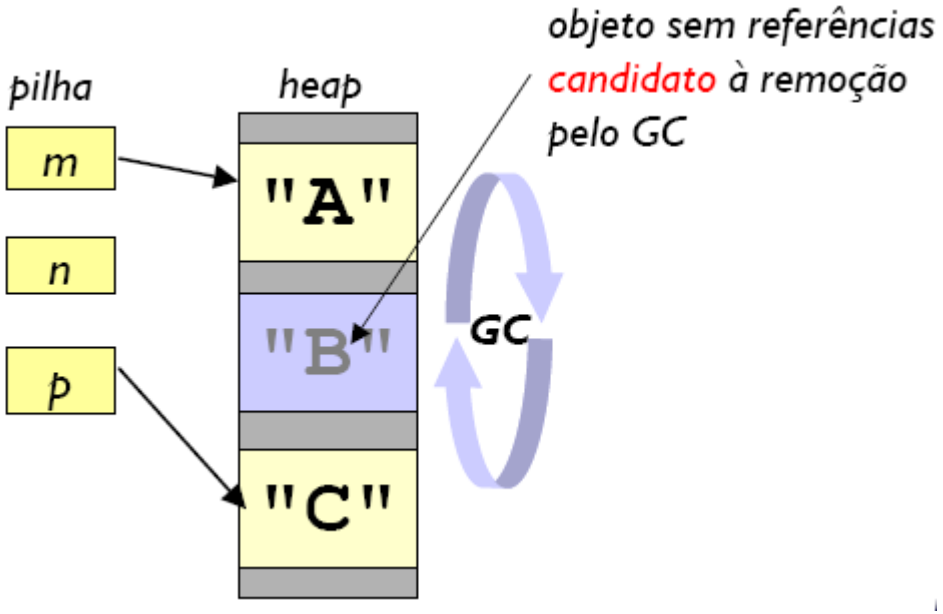


Coleta de lixo

- *Quando um objeto não tem mais referências apontando para ele, seus dados não mais podem ser usados, e a memória deve ser liberada.*
- *O coletor de lixo irá liberar a memória na primeira oportunidade*

```
n = null;

p = new Mensagem("C");
```



O que Java não faz

- *Java não suporta herança múltipla de implementação*
  - *Herança múltipla é característica comum a várias linguagens OO, e permite reuso de código de várias classes em outra classe*
  - *Tem vantagens porém aumenta a complexidade*
  - *Java oferece uma solução que preserva as principais vantagens da herança múltipla e evita os problemas*
- *Java não suporta aritmética de ponteiros*
  - *Ponteiros, ou referências, são usados em várias linguagens, inclusive Java, para manipular eficientemente grandes quantidades de informação na memória*
  - *Com ponteiros, em vez de copiar uma informação de um lugar para outro, copia-se apenas o seu endereço*
  - *Em linguagens como C, o programador pode manipular o endereço (que é dependente de plataforma) diretamente*
  - *Isto aumenta a complexidade e diminui a portabilidade*

O JDK

- *O JDK (Java Development Kit) é o ambiente padrão distribuído pela Sun para desenvolvimento de aplicações Java*
- *O JDK consiste de*
  - *JRE (Java Runtime Environment) - também distribuído separadamente: ambiente para execução de aplicações*
  - *Ferramentas para desenvolvimento: compilador, debugger, gerador de documentação, empacotador JAR, etc.*
  - *Código-fonte das classes da API*
  - *Demonstrações de uso das APIs, principalmente Applets, interface gráfica com Swing e recursos de multimídia*
- *A documentação é distribuída separadamente*

Como compilar

- *Use o java compiler (linha de comando)*
  - **javac** NomeDaClasse.java
  - **javac -d** ../destino Um.java Dois.java
  - **javac -d** ../destino *.java
  - **javac -classpath** c:\fontes -d ../destino *.java
- *Algumas opções (opcionais)*
  - **-d** *diretório onde serão armazenadas as classes (arquivos .class) geradas*
  - **-classpath** *diretórios (separados por ; ou :) onde estão as classes requeridas pela aplicação*
  - **-sourcepath** *diretórios onde estão as fontes*
- *Para conhecer outras opções do compilador, digite **javac** sem argumentos*
- *Compiladores de outros fabricantes (como o **JCreator)** também podem ser usados no lugar do javac*

Como executar

- *Use o interpretador java (faz parte do JRE)**
  - *java NomeDaClasse*
  - *java pacote.subpacote.NomeDaClasse*
  - *java -classpath c:\classes;c:\bin;. pacote.Classe*
  - *java -cp c:\classes;c:\bin;. pacote.Classe*
  - *java -cp %CLASSPATH%;c:\mais pacote.Classe*
  - *java -cp biblioteca.jar pacote.Classe*
  - *java -jar executavel.jar*
- *Para rodar aplicações gráficas, use javaw*
  - *javaw -jar executavel.jar*
  - *javaw -cp aplicacao.jar;gui.jar principal.Inicio*
- *Principais opções*
  - *-cp ou -classpath classpath novo (sobrepõe v. ambiente)*
  - *-jar executa aplicação executável guardada em JAR*
- *-Dpropriedade=valor define propriedade do sistema (JVM)*

» ** sintaxe de PATH em Unix é diferente*

Algumas outras ferramentas do SDK

- *Debugger:* ***jdb***
  - *Depurador simples de linha de comando*
- *Profiler:* ***java –prof***
  - *Opção do interpretador Java que gera estatísticas sobre uso de métodos em um arquivo de texto chamado java.prof*
- *Java Documentation Generator:* ***javadoc***
  - *Gera documentação em HTML (default) a partir de código-fonte Java*
- *Java Archiver:* ***jar***
  - *Extensão do formato ZIP; ferramenta comprime, lista e expande*
- *Applet Viewer:* ***appletviewer***
  - *Permite a visualização de applets sem browser*
- *HTML Converter:* ***htmlconverter.jar***
  - *Converte <applet> em <object> em páginas que usam applets*
- *Disassembler:* ***javap***
  - *Permite ler a interface pública de classes*

Parte prática

Introdução prática

- *Nesta seção serão apresentados alguns exemplos de aplicações simples em Java*
  - *1. Aplicação HelloWorld*
  - *2. Aplicação HelloWorld modificada para promover reuso e design orientado a objetos (duas classes)*
  - *3. Aplicação Gráfica Swing (três classes)*
  - *4. Aplicação para cliente Web (applet)*
- *Compilar código em casa*
  - *Apresentação 1.*
- *Todos os assuntos apresentados nesta seção serão explorados em detalhes em aulas posteriores*
  - *Conceitos como classe, objeto, pacote*
  - *Representação UML*
  - *Sintaxe, classes da API, etc.*

Aplicação HelloWorld

- *Esta mini-aplicação em Java imprime um texto na tela quando executada via linha de comando.*

```java
/** Aplicação Hello World */
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, world!");
    }
}
```

HelloWorld.java

- *Exercício: Use-a para testar seu ambiente e familiarizar-se com o desenvolvimento Java*
  - *Digite-a no seu editor de textos*
  - *Tente compilá-la*
  - *Corrija eventuais erros*
  - *Execute a aplicação*

Anatomia

Comentário de bloco

Nome da classe

Nome do método

Declaração de argumento

variável local: `args`

tipo: `String[]`

```
/** Aplicação Hello World */
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, world!");
    }
}
```

Ponto-e-vírgula é obrigatório no final de toda instrução

Definição de classe `HelloWorld`

Definição de método `main()`

Chamada de método `println()` via objeto `out` accessível através da classe `System`

Atribuição de argumento para o método `println()`

Uma classe define um **tipo de dados**

- *Esta classe representa objetos que guardam um texto (tipo String) em um atributo (**msg**) publicamente acessível.*
- *Além de guardar um String, retorna o texto em caixa-alta através do método **lerNome().***

```
public class Mensagem {
    public String msg = "";
    public String lerNome() {
        String nomeEmMaiusculas =
                    msg.toUpperCase();
        return nomeEmMaiusculas;
    }
}
```

atributo

método

Membros da classe. Outras classes podem acessá-los, se declarados como "public", usando o operador ponto "."

Representação em UML

Mensagem
+msg: String
+lerNome(): String

*Esta é a **interface pública** da classe. É só isto que interessa a quem vai usá-la. Os detalhes (código) estão **encapsulados**.*

Classe executável que usa um tipo

- *Esta outra classe **usa** a classe anterior para criar um objeto e acessar seus membros visíveis por sua **interface pública***
  - *Pode alterar ou ler o valor do atributo de dados **msg***
  - *Pode chamar o método **lerNome()** e usar o valor retornado*

```
public class HelloJava {
  private static Mensagem nome;

  public static void main(String[] args) {
    nome = new Mensagem();    // cria objeto

    if (args.length > 0) {    // há args de linha de comando?
       nome.msg = args[0];    // se houver, copie para msg
    } else {
       nome.msg = "Usuario"; // copie palavra "Usuario"
    }

    String texto = nome.lerNome(); // chama lerNome()
    System.out.println("Bem-vindo ao mundo Java, "+texto+"!");
  }
}
```

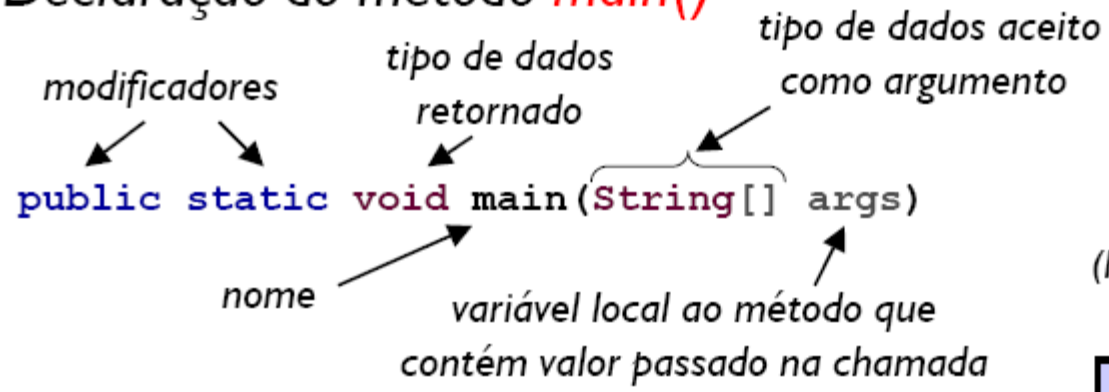
atributo nome é do tipo Mensagem

Este método é chamado pelo interpretador

Operador de concatenação

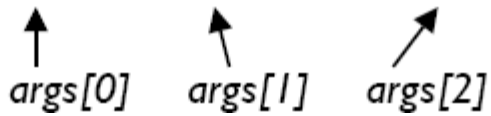
Detalhes

- Declaração do método main()

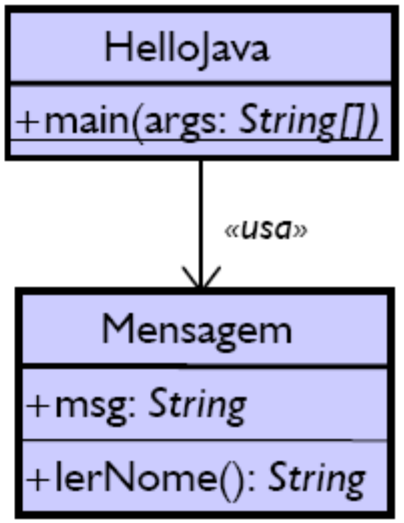


- O método main() é chamado pelo interpretador Java, automaticamente
  - Deve ter sempre a assinatura acima
- O argumento é um vetor formado por textos passados na linha de comando:

```
> java NomeDaClasse Um "Dois Tres" Quatro
```



Dependência entre as duas classes (HelloJava tem referência para Mensagem)



Primeira aplicação gráfica

- *A aplicação abaixo cria um objeto do tipo JFrame (da API Swing) e reutiliza a classe Mensagem*

```
import javax.swing.*; // importa JFrame e JLabel
import java.awt.Container;

public class MensagemGUI {
  public MensagemGUI(String texto) {
    JFrame janela = new JFrame("Janela");
    Container areaUtil = janela.getContentPane();
    areaUtil.add( new JLabel(texto) );
    janela.pack();
    janela.setVisible(true);
  }
}
```

Quando objeto é criado, construtor MensagemGUI é chamado.

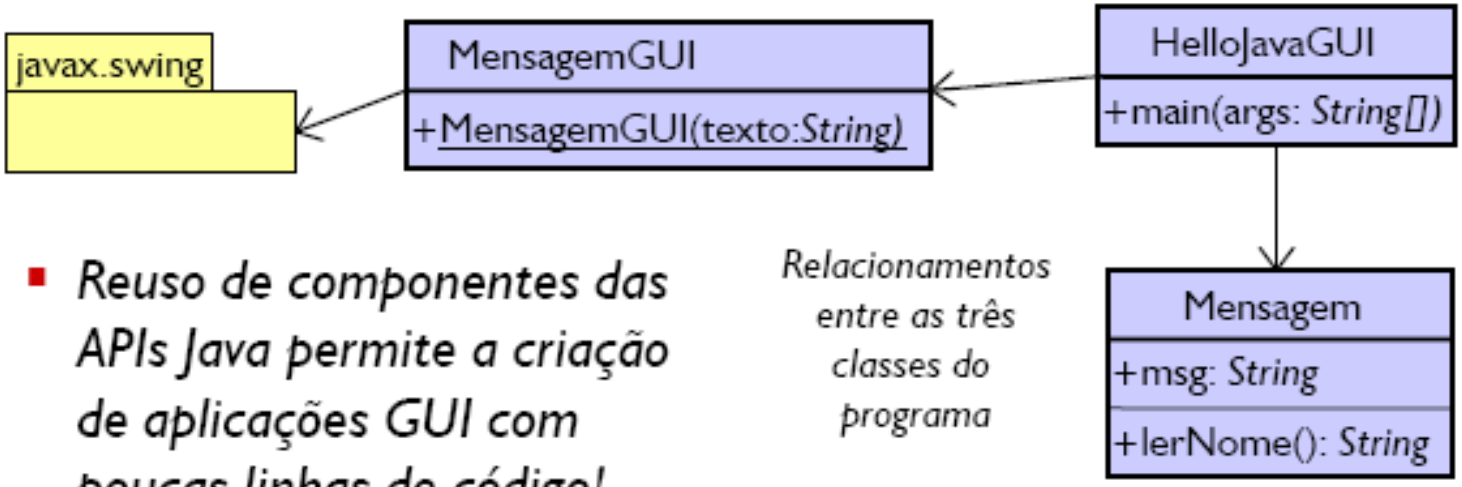
Construtor cria janela contendo texto recebido

```
public class HelloJavaGUI {
  private static Mensagem nome;
  public static void main(String[] args) {
    (... igual a HelloJava ...)
    String texto = nome.lerNome();
    new MensagemGUI
        ("Bem-vindo ao mundo Java, "+texto+"!");
  }
}
```

reuso!

No lugar de imprimir o texto, passa-o como parâmetro na criação de MensagemGUI

Componentes da aplicação gráfica

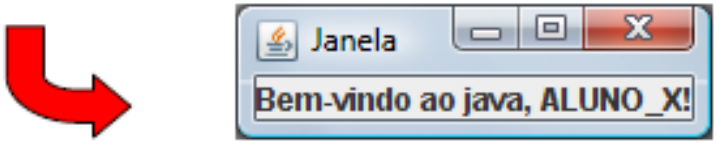


- *Reuso de componentes das APIs Java permite a criação de aplicações GUI com poucas linhas de código!*

Relacionamentos entre as três classes do programa

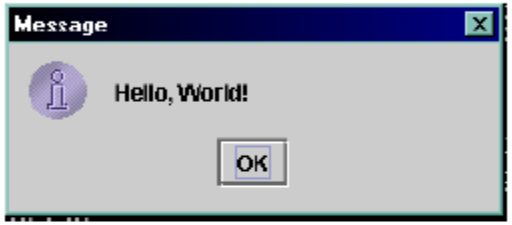
```
java_work>java HelloJavaGUI "Aluno_X"
```

Execução da apliação passando parâmetro via linha de comando



Entrada de dados via GUI

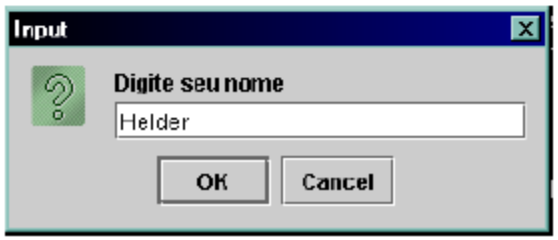
- *javax.swing.JOptionPane oferece uma interface gráfica para entrada de dados e exibição de informações*
- *Exemplo de exibição de caixa de diálogo*
  - `JOptionPane.showMessageDialog(null, "Hello, World!");`



É preciso importar javax.swing. ou javax.swing.JOptionPane*

- *Exemplo de diálogo para entrada de dados*

```
String nome =
    JOptionPane.showInputDialog("Digite seu nome");
if (nome != null) {
    JOptionPane.showMessageDialog(null, nome);
} else {
    System.exit(0);
}
```



Primeiro applet

- *Componentes gráficos que podem ser executados no browser*
- *Para criar e usar um applet é preciso*
  - *criar uma classe que herde da classe Applet ou JApplet (API Java)*
  - *criar uma página HTML que carregue o applet*
- *A classe abaixo implementa um JApplet*

```
import javax.swing.*; // importa JFrame e JLabel
import java.awt.Container;

public class HelloJavaApplet extends JApplet {
  private Mensagem nome;

  public void init() {
    nome = new Mensagem();
    nome.msg = this.getParameter("texto"); // parâmetro HTML
    String texto = nome.lerNome();
    Container areaUtil = this.getContentPane();
    JLabel label =
       new JLabel("Bem-vindo ao mundo Java, " +texto+ "!");
    areaUtil.add(label);
  }
}
```

Herança!

Chamado automaticamente pelo browser

Página HTML

- *O elemento <applet> é usado para incluir applets antigos (Java 1.0 e 1.1) em páginas HTML ou servir de template para a geração de código HTML 4.0*

```
<html>
 <head>
  <title>Sem Título</title>
 </head>
 <body>
  <h1>Um Applet</h1>
  <applet code="HelloJavaApplet.class" height="50" width="400">
    <param name="texto" value="Helder">
  </applet>
 </body>
</html>
```

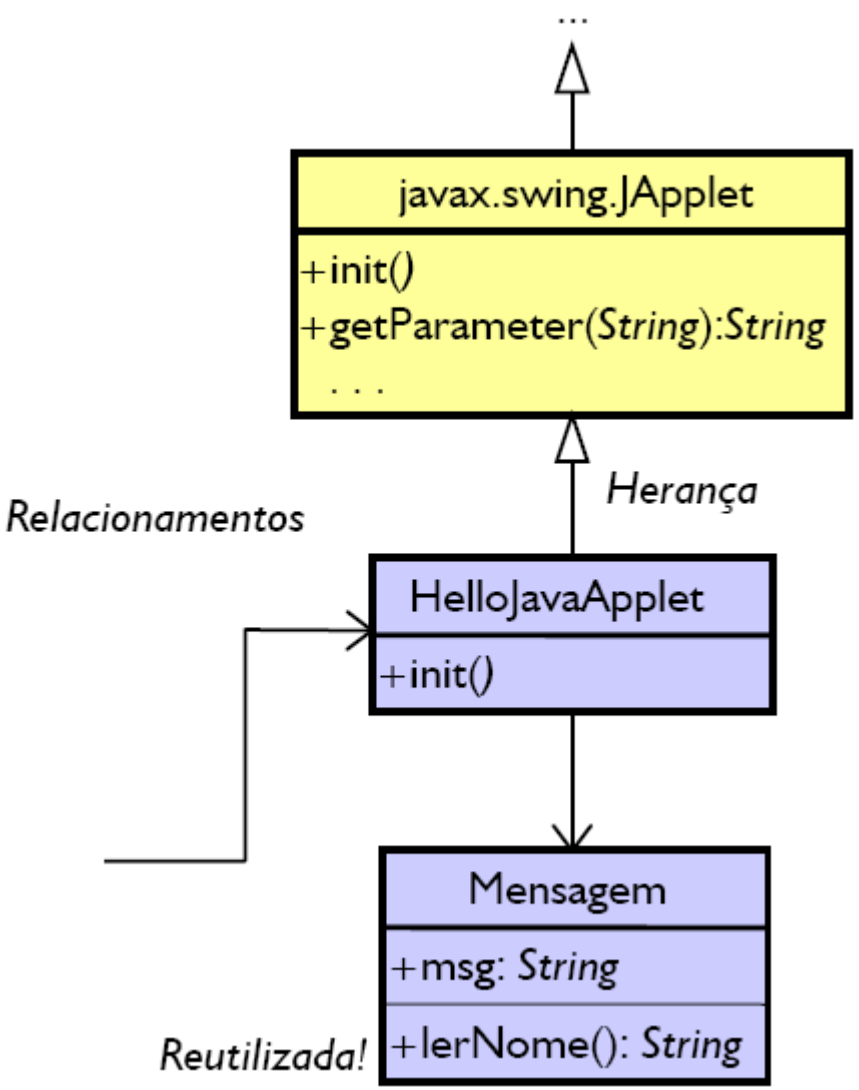
- *Converta o código para HTML 4.0: ferramenta htmlconverter*
  - *Guarde uma cópia do original, e rode (use htmlconverter.bat)*
- > `htmlconverter pagina.html`

Detalhes

- *Para rodar o applet abra a página com seu browser ou use o appletviewer*

  > ***appletviewer*** *pagina.html*

- *Mude o valor dos parâmetros do HTML e veja os resultados*



Um Applet

Seja bem-vindo ao mundo Java, HELDER!

Resumo

- *Foram apresentados quatro exemplos de pequenas aplicações Java, demonstrando*
  - *Sintaxe elementar, compilação e execução*
  - *Classe como unidade de código para execução*
  - *Classe como definição de tipo de dados*
  - *Reuso de objetos através de associação (uso da classe Mensagem em três aplicações diferentes) e herança (infraestrutura de Applets reaproveitada)*
  - *Aplicações gráficas*
  - *Componentes de um framework (Applets) que executam em um container padrão (dentro do browser)*

Referências

- Adaptado de Helder da Rocha, **J100: Programação Orientada a Objetos usando Java**. www.argonavis.com.br
- Deitel & Deitel. **Java como programar**, 6 ed.
- Kathy Sierra e Bert Bates. **Use a cabeça Java!.**